Heuristic Graph Pathfinding Agent Search Engine

AI/ML Internship — Task 3 Report

Field	Details
Task	Task 3 — Heuristic Graph Pathfinding Agent Search Engine
Domain	Artificial Intelligence / Search Algorithms
Programming Language	Python
Algorithms	A* Search and Dijkstra's Algorithm
Heuristic	Manhattan distance
Grid Movement	Four-directional, unit movement costs
Repository	github.com/MuhammadAhmedSubhani/Heuristic-Graph-Pathfinding-Agent-Search-Engine

Project objective: Build an optimization search framework that navigates grid mazes, compares heuristic and non-heuristic pathfinding algorithms, records metrics such as path length, runtime and expanded nodes, and exports visualization logs showing search behavior.

Final automated test result: 3 tests passed

1. Introduction

This project implements and compares two classical graph-search algorithms for navigating grid-based mazes: Dijkstra's algorithm and A* Search. Dijkstra explores nodes according to the lowest known path cost, while A* combines the path cost so far with a heuristic estimate of the remaining distance to the goal.

The framework evaluates both algorithms across six maze layouts, including a larger 10×10 grid and a maze whose goal is unreachable. It records path lengths, expanded-node counts, node-expansion density and runtime, and writes benchmark and visualization outputs to files.

2. Problem Definition and Objectives

The agent must find a valid minimum-cost route from a start cell to a goal cell while avoiding blocked cells. If no route exists, the algorithm must report that the goal is unreachable. The project objectives are:

• Implement A* Search and Dijkstra's algorithm from scratch.

• Use a consistent grid representation and four-directional movement with unit edge costs.

• Evaluate both algorithms across different obstacle arrangements.

• Compare shortest-path results, expanded nodes, expansion density and execution time.

• Export machine-readable benchmark results and a text visualization log.

3. Algorithms and Methodology

3.1 Dijkstra's Algorithm

Dijkstra's algorithm maintains a priority queue ordered by the accumulated path cost from the start. It repeatedly expands the lowest-cost frontier node and updates neighboring distances when a lower-cost route is found. With non-negative edge costs, it finds a shortest path when one exists.

3.2 A* Search

A* prioritizes nodes using f(n) = g(n) + h(n), where g(n) is the known cost from the start and h(n) estimates the remaining cost to the goal. This project uses Manhattan distance for four-directional grid movement: $h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$. For this grid setup, the heuristic does not overestimate the true remaining distance, supporting optimal-path search.

3.3 Experimental Setup

Setting	Implementation
Search space	Six grid mazes with varying layouts and obstacles
Movement	Up, down, left and right
Movement cost	1 per move
Priority queue	Python heapq
Runtime measurement	Python time.perf_counter
Recorded metrics	Path length, expanded nodes, expansion density, runtime, node reduction

4. Benchmark Results

The benchmark results below compare shortest-path length and the number of expanded nodes. The path-length column reports the common shortest-path length on reachable mazes; Maze 6 has no valid route.

Maze	Shortest Path	Dijkstra Expanded	A* Expanded	A* Node Reduction
Maze 1 — Simple	8	19	17	10.53%
Maze 2 — More Obstacles	8	19	19	0.00%
Maze 3 — Dense Obstacles	8	18	16	11.11%
Maze 4 — Larger 10×10 Grid	18	61	49	19.67%
Maze 5 — Different Layout	8	15	11	26.67%
Maze 6 — No Path	No path	11	11	0.00%

4.1 Interpretation

• **Shortest-path correctness:** A* and Dijkstra returned equal shortest-path lengths in all five reachable mazes.

• **Node expansion:** A* expanded fewer nodes in four of the six test cases.

• **Largest reduction:** Maze 5 had a 26.67% reduction in expanded nodes (15 for Dijkstra versus 11 for A*).

• **Larger grid:** On Maze 4, A* expanded 49 nodes compared with 61 for Dijkstra, a 19.67% reduction.

• **Unreachable goal:** Both algorithms correctly reported no path for Maze 6.

4.2 Node-Expansion Comparison

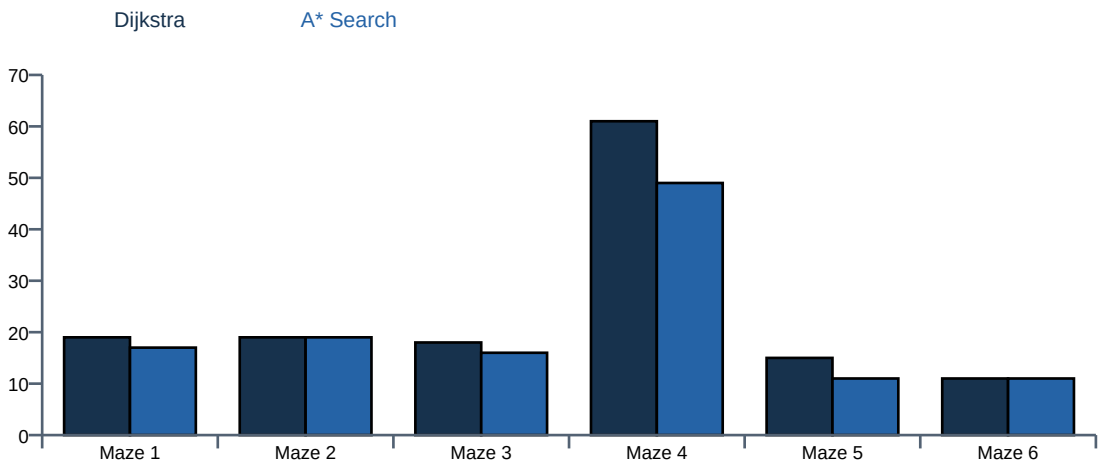

Figure 1. Expanded nodes by maze. Lower counts indicate that fewer nodes were expanded during search; the chart does not imply that A* is always faster in wall-clock time.

5. Metrics and Calculations

5.1 Path Length

Path length is the number of moves in the returned route. Both algorithms use unit movement costs, so equal path lengths also indicate equal route costs for the reachable mazes.

5.2 Expanded-Node Reduction

The relative reduction in expanded nodes is calculated against Dijkstra's count:

> **Node reduction (%) = ((Dijkstra expanded − A* expanded) / Dijkstra expanded) × 100**

5.3 Runtime and Expansion Density

Runtime measurements are saved in results/benchmark_results.csv. Because these searches complete in very small time intervals, runtime can vary with machine load and measurement overhead. Runtime should therefore be treated as illustrative rather than as a stable performance claim. Expansion density records the expanded search effort relative to the relevant grid size, as implemented by the project.

6. Exported Results and Visualization

Output file	Purpose
results/benchmark_results.csv	Structured metrics for all six maze cases, including path, expanded nodes, density, runtime and node reduction.
results/maze_4_visualization.txt	Text-based search visualization for Maze 4, the larger 10×10 grid.
reports/benchmark_summary.md	Readable summary of the benchmark methodology, results and findings.

These files make it possible to inspect the algorithm comparison outside the program and provide a reproducible record of the experiment outputs generated by the project.

7. Automated Testing

Test	Result
test_a_star_finds_a_valid_shortest_path	Passed
test_a_star_matches_dijkstra_on_reachable_mazes	Passed
test_both_algorithms_report_unreachable_goal	Passed

Verified test run: `python -m unittest discover -s tests -v` was executed using the project's virtual environment interpreter. All 3 tests passed in 0.022 seconds.

8. Project Structure

```
Heuristic-Graph-Pathfinding-Agent-Search-Engine/
■■■ app.py
■■■ src/
■ ■■■ __init__.py
■ ■■■ project.py
■■■ tests/
■ ■■■ test_pathfinding.py
■■■ results/
■ ■■■ benchmark_results.csv
■ ■■■ maze_4_visualization.txt
■■■ reports/
■ ■■■ benchmark_summary.md
■■■ README.md
■■■ LICENSE
■■■ .gitignore
```

9. Technologies Used

Category	Technology / Component
Programming language	Python
Priority queue	heapq
Timing	time.perf_counter
Testing	unittest
Data export	CSV and text files
Version control	Git and GitHub

10. Limitations and Future Improvements

• The experiments use a limited set of six grid layouts; larger and randomly generated maps would provide broader evidence.

• The current movement model uses four directions and uniform movement costs; weighted terrain and diagonal movement are not evaluated.

• Wall-clock runtime is sensitive to environment and measurement noise, especially for very small searches.

• Future work could add randomized maze generation, repeated benchmark trials with averages and dispersion, and richer visualizations.

11. Conclusion

The project delivers a working grid-maze search framework that compares A* Search with Dijkstra's algorithm. Across five reachable mazes, both algorithms found equal-length shortest paths, while A* expanded fewer nodes in four of the six total cases. The largest measured reduction was 26.67% on Maze 5, and A* reduced expansions by 19.67% on the larger 10×10 Maze 4. Both algorithms also correctly identified the unreachable goal in Maze 6.

The project exports benchmark metrics and a text visualization log, includes a written benchmark summary, and has passed all three automated tests. The results support the value of heuristic guidance in reducing node expansion on these test cases, while runtime claims remain limited by the small experiment scale.

GitHub repository: https://github.com/MuhammadAhmedSubhani/Heuristic-Graph-Pathfinding-Agent-Search-Engine